Intelligent and Communicating Systems (ICS)

LAB N°02 – Arduino Communications (GPIO–Sensors–Actuators)

Full Name: Amel Sadoun – Group 2SQ2

A. THEORY

1. Pushbutton and Digital GPIO Pins

A **pushbutton** is a digital input device that provides binary states (pressed or released). On the Arduino MKR1010, **digital GPIO pins** can be configured as input or output using the `pinMode()` function. When configured as `INPUT`, the pin reads a voltage level: HIGH (logic 1) or LOW (logic 0). When configured as `OUTPUT`, it drives external components such as LEDs or relays.

2. Analog GPIO Pins of MKR1010

The MKR1010 board includes several analog GPIO pins (`A0{A6`) that can read continuous voltage values (0–3.3 V) using a 10-bit ADC (Analog-to-Digital Converter). The function `analogRead()` returns an integer between 0 and 1023, representing the sensor voltage level. Analog outputs can be simulated using `analogWrite()` with PWM (Pulse Width Modulation).

3. Debounce Circuit and Its Necessity

A mechanical pushbutton does not change state cleanly when pressed or released. It produces small voltage oscillations known as **bouncing**. A **debounce circuit** (typically a resistor-capacitor filter) or a software delay removes these fluctuations, ensuring that a single press is detected as one logical event. Debouncing is necessary for accurate input readings.

4. Pull-Up and Pull-Down Resistors

A **pull-up resistor** connects the input pin to a high voltage level when the button is open, while a **pull-down resistor** connects it to ground. These resistors prevent the input pin from floating, which can cause unstable or random readings. Arduino provides an internal pull-up resistor that can be activated using `pinMode(pin, INPUT_PULLUP)`.

5. Light Sensor (LDR) and Analog Input

A **Light-Dependent Resistor (LDR)** changes its resistance according to the surrounding light intensity. When connected as part of a voltage divider, the resulting voltage is measured on an analog GPIO pin. The Arduino Serial Monitor can then display this value to visualize light variations.

B. ACTIVITY – PART 01

1. Light Sensor (Analog Input)

Objective: Read the ambient light level using an LDR connected to analog pin A0 and display the value on the Serial Monitor.

Circuit:

- LDR connected between 3.3 V and A0.

- A fixed 10 kΩ resistor connected between A0 and GND (voltage divider).

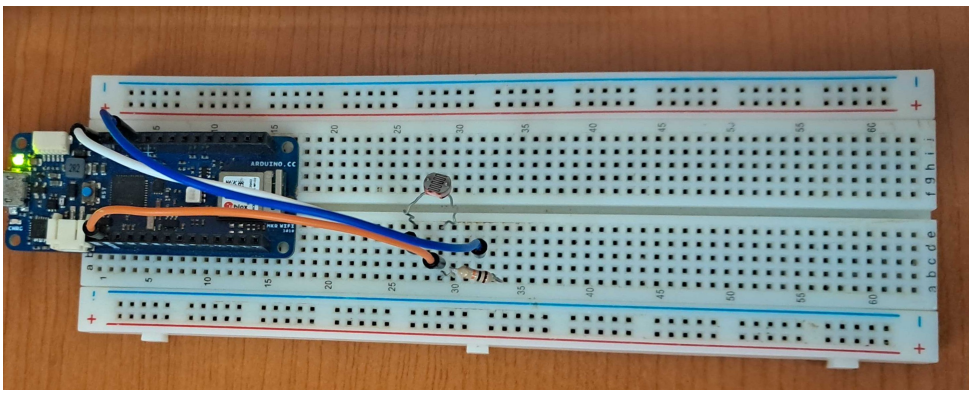

Figure 1 – LDR circuit connection to A0

Code Example:

Listing 1: Reading LDR value from analog pin A0

```
int ldrPin = A0;    // Analog input pin connected to LDR
int ldrValue = 0;   // Variable to store sensor value

void setup() {
  Serial.begin(9600);  // Start serial communication
}

void loop() {
  ldrValue = analogRead(ldrPin);    // Read the sensor value
  Serial.print("Light Level: ");
  Serial.println(ldrValue);         // Display it on Serial Monitor
  delay(500);                       // Wait half a second
}
```

Explanation: The LDR voltage is sampled and displayed every 500 ms. Higher readings indicate brighter light. This data can later be used to control an LED or actuator automatically.

2. Pushbutton (Digital Input with Pull-Down Resistor)

Objective: Detect the pushbutton state and display it on the Serial Monitor using an **external pull-down resistor** to ensure stable readings.

Circuit Description:

- The pushbutton is connected between 3.3 V and digital pin 2.
- A 10 kΩ resistor connects pin 2 to GND, acting as a pull-down resistor.

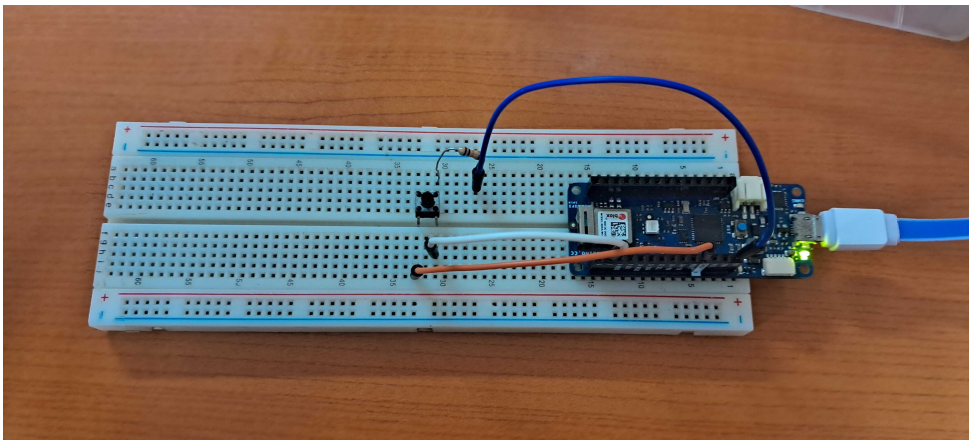

Figure 2 – External pushbutton with pull-down resistor keeping the pin LOW when idle

Code Example (External Pull-Down):

Listing 2: Pushbutton input using external pull-down resistor

```
const int buttonPin = 2;  // Pushbutton connected to pin 2
int buttonState = 0;

void setup() {
  pinMode(buttonPin, INPUT);   // Using external pull-down resistor
  Serial.begin(9600);
}

void loop() {
  buttonState = digitalRead(buttonPin);

  if (buttonState == HIGH) {   // Button pressed (voltage pulled HIGH)
    Serial.println("Button pressed");
  } else {                     // Button released (voltage LOW)
    Serial.println("Button released");
  }

  delay(200);  // Small delay for readability
}
```

Explanation: In this configuration, the external pull-down resistor ensures that the input pin stays at logic LOW when the pushbutton is not pressed, preventing floating inputs. When the button is pressed, the circuit connects the pin directly to 3.3 V, resulting in a HIGH reading. This setup corresponds exactly to the wiring shown in Figure 2. Adding a small capacitor in parallel with the button later forms a **hardware debounce circuit**, which can be tested in the next section to observe cleaner signal transitions.

3. External Pushbutton with LED Toggle

Objective: Use an external pushbutton on pin 7 to toggle an LED on pin 2 each time the button is pressed.

Code Example:

Listing 3: External pushbutton controlling an LED

```
const int buttonPin = 7;   // Pushbutton pin
const int ledPin = 2;      // LED pin

int ledState = LOW;        // LED initial state
int lastButtonState = HIGH; // Previous button state
int currentButtonState;

void setup() {
  pinMode(ledPin, OUTPUT);       // Set LED pin as output
  pinMode(buttonPin, INPUT);     // Use external resistor circuit
  Serial.begin(9600);
}

void loop() {
  currentButtonState = digitalRead(buttonPin);

  // Detect button press (transition from HIGH to LOW)
  if (lastButtonState == HIGH && currentButtonState == LOW) {
```

```
    ledState = (ledState == LOW) ? HIGH : LOW; // Toggle LED state
    digitalWrite(ledPin, ledState);
    Serial.println("Button pressed!");
    delay(200); // Simple debounce delay
  }

  lastButtonState = currentButtonState;
}
```

Explanation: Every button press toggles the LED between ON and OFF. The short delay acts as a simple software debounce.

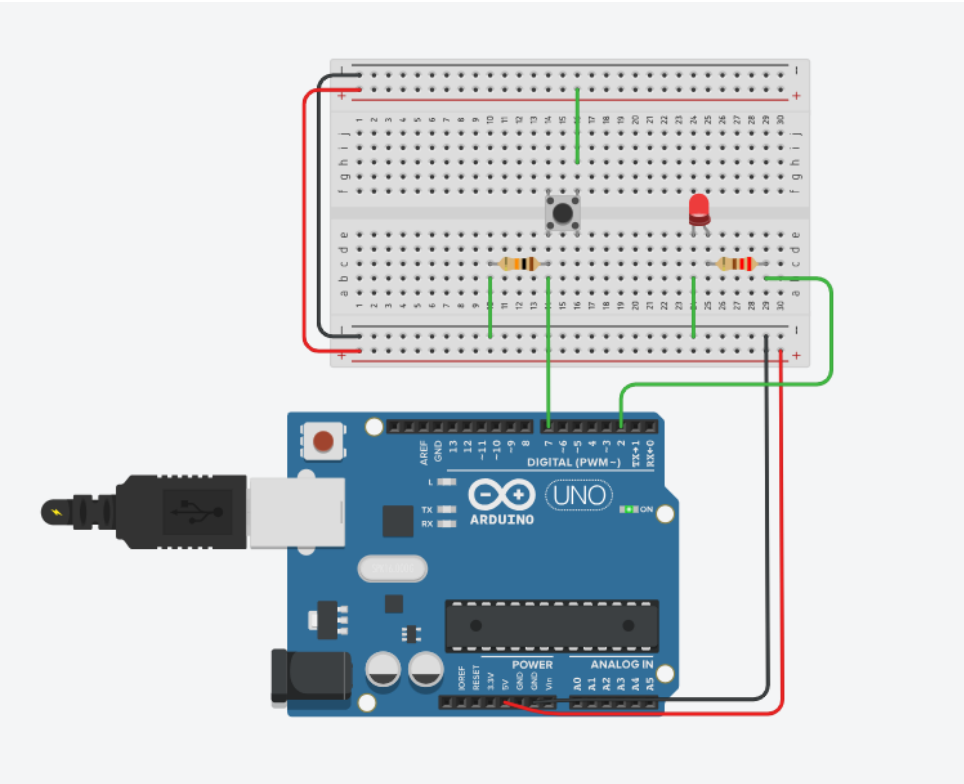

Figure 3 – External button controlling LED state

4. Debounce Circuit Simulation

Objective: Observe the effect of switch bouncing and verify the improvement when using a debounce circuit.

Circuit Description: A capacitor (100 nF) is connected in parallel with the pushbutton, forming an RC low-pass filter that smooths out voltage spikes. The resistor ensures controlled discharge of the capacitor between presses.

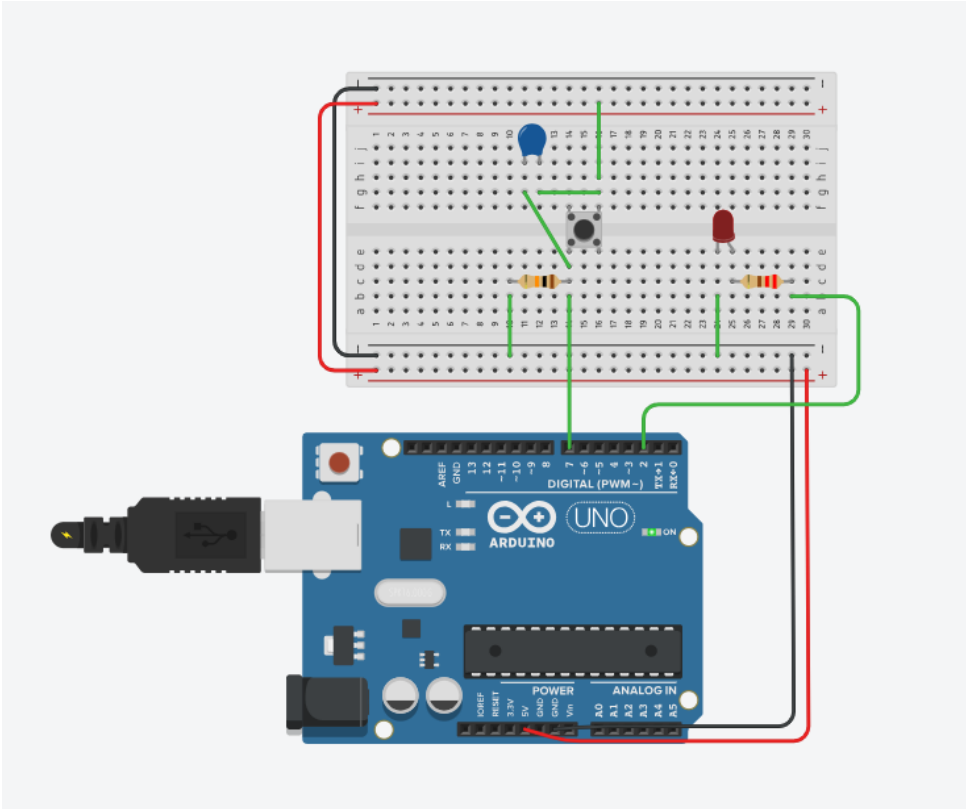

Figure 4 – Pushbutton with hardware debounce circuit

Observed Result: Without a capacitor, pressing the button causes multiple rapid transitions between 0 and 1 due to mechanical bounce. Adding the capacitor stabilizes the voltage, producing only one clean transition per press. Therefore, the debounce circuit is essential for stable digital input detection.

B. ACTIVITY – PART 02

5. Using LED with LDR and Pushbutton

Objective: Combine the light sensor and pushbutton to control an LED based on both light level and manual input.

Circuit Overview:

- LDR on analog input A0 (light detection)
- Pushbutton on digital pin 2 (manual override)
- LED on PWM pin 9 (brightness control)

Code Example (Smart Light Control):

Listing 4: LED control based on LDR and pushbutton state

```
int ldrPin = A0;
int ledPin = 9;
int buttonPin = 2;
int ldrValue;
int buttonState;

void setup() {
```

```
  Serial.begin(9600);
  pinMode(ledPin, OUTPUT);
  pinMode(buttonPin, INPUT_PULLUP);
}

void loop() {
  ldrValue = analogRead(ldrPin);
  buttonState = digitalRead(buttonPin);

  if (buttonState == LOW) {
    digitalWrite(ledPin, HIGH); // Manual override ON
    Serial.println("Manual mode: LED ON");
  } else {
    if (ldrValue < 400) {
      int brightness = map(ldrValue, 0, 400, 255, 100);
      analogWrite(ledPin, brightness);
      Serial.println("Auto mode: LED dimmed");
    } else {
      digitalWrite(ledPin, LOW);
      Serial.println("Auto mode: LED OFF");
    }
  }
  delay(500);
}
```

Explanation: This system turns the LED ON in darkness (LDR ¡ 400) and OFF in bright light. The pushbutton overrides automatic mode, forcing the LED to stay ON. PWM enables gradual brightness control according to light intensity, creating a basic **smart streetlight simulation** suitable for IoT applications.

C. CONCLUSION

This lab provided a deeper understanding of microcontroller GPIO management through hands-on experiments. We learned to:

- Configure and read both digital and analog GPIO lines of the Arduino MKR1010.
- Apply pull-up/pull-down resistors and debounce mechanisms for stable digital inputs.
- Interface sensors (LDR) and actuators (LEDs) to build responsive systems.
- Combine analog sensing and digital control to design intelligent behaviors such as automatic light regulation.